UART FAQ Troubleshooting Guide

ID: RK-PC-YF-164

Release Version: V1.2.0

Release Date: 2026-06-22

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

Building 18, Area A, Software Park, Tongpan Road, Fuzhou, Fujian, P.R.C.

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides solutions to common issues encountered during UART development and maintenance, and lists resolved UART issues on Rockchip chip platforms as references. Readers can locate corresponding solutions based on UART issue classifications.

Product Versions

Chip Models	Kernel Version
All chips using the Linux kernel	Linux Kernel

Target Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineer

Software Development Engineer

Revision History

Version	Author	Date	Modification Description
V1.0.0	Liu Shifang	2021-05-21	Initial version
V1.1.0	Liu Shifang	2021-12-29	Updated version
V1.2.0	Caesar Wang	2026-06-22	Fixed clock source threshold contradiction, DTS syntax errors, terminology, formatting, and cross-reference issues

Table of Contents

UART FAQ Troubleshooting Guide

- 1. Usage Instructions
  - 1.1 Related Documents
  - 1.2 Boot Logs
  - 1.3 io Command
  - 1.4 Disable Other Print Interference
- 2. Baud Rate Related
  - 2.1 Clock Prescaling Strategy
  - 2.2 Cases
    - 2.2.1 Incorrect Baud Rate Measurement Method
    - 2.2.2 Errors Occur During Operation but Resolve After Restart
    - 2.2.3 Abnormal Behavior When Switching from High to Low Baud Rate
- 3. Pin Multiplexing Related
  - 3.1 Cases
    - 3.1.1 TX Pins of Different IOMUX Groups Can Be Used Simultaneously, But RX Pins Cannot
    - 3.1.2 UART Malfunction After Enabling Other Modules
- 4. Pin Level Issues
  - 4.1 Cases
    - 4.1.1 Hardware PCB Design Errors
    - 4.1.2 Inconsistency Between Hardware Design and Actual Chip Model
    - 4.1.3 Peripheral TX Unable to Pull Down Main Controller RX
- 5. Interrupt-Related Issues
  - 5.1 Cases
    - 5.1.1 Configuring the Receive Interrupt Trigger Watermark
- 6. DMA Related
  - 6.1 Cases
    - 6.1.1 DMA Configurations May Differ Across Kernel Versions
    - 6.1.2 Probabilistic Data Loss During DMA-mode Reception at High Baud Rates
- 7. Hardware Flow Control
  - 7.1 Cases
    - 7.1.1 Unable to Transmit Data After Enabling Hardware Flow Control
- 8. Data Reception Errors
  - 8.1 Cases
    - 8.1.1 Incorrect Data Transmission from the Counterpart
    - 8.1.2 Data Segmentation During Large-Volume Reception
    - 8.1.3 Latency in Application-Layer Data Reading
- 9. Issues Related to Connecting Peripherals or Other Controllers
  - 9.1 Cases
    - 9.1.1 Bluetooth Transmission Anomalies
    - 9.1.2 Abnormalities When Using RS485
- 10. Console Printing
  - 10.1 Cases
    - 10.1.1 Shell Hangs When Sending Large Numbers of Test Commands via Console
    - 10.1.2 Carriage Return and Line Feed Issues
- 11. suspend, resume, and Power Consumption
- 12. Disable All Prints or Redirect All Prints to Another UART
  - 12.1 DDR Loader Modification Method
  - 12.2 U-Boot Modification Method
  - 12.3 Kernel Modification Method
    - 12.3.1 Disabling Print Output in the Kernel
    - 12.3.2 Switching Kernel Print Output

12.3.3 Using the Original Console UART as a Standard UART
12.4 Android Modification Method
12.5 Cases
12.5.1 Issues Arising After Switching from a Logging UART to a Standard UART
13. Other Issues
13.1 Cases
13.1.1 Kernel Fails to Boot Normally with Certain Host Tools
13.1.2 Changing UART Device Node Numbers

1. Usage Instructions

1.1 Related Documents

Before using this document, please read the following documents to ensure a thorough understanding of Rockchip UART features, usage methods, and debugging techniques:

- *Rockchip_Developer_Guide_UART_EN*: Rockchip Linux Kernel UART Developer Guide.
- UART section in *Rockchip_Developer_Guide_UBoot_Nextdev_CN*: Rockchip U-Boot UART Developer Guide.

Depending on the usage scenario and specific hardware/software platform, solutions for identical issues may vary; please handle them flexibly. Trust the robustness of the basic UART functions in the Rockchip platform SDK; the basic data transmission and reception functions of each UART have been tested. If you are using an older SDK version, please obtain the latest SDK source code and compare the UART Driver code. If you have any questions, please contact FAE via the Rockchip Redmine platform for technical support.

1.2 Boot Logs

Based on the characteristic printouts during each boot stage in the boot logs, the stage where the UART issue occurs can be identified, allowing for targeted troubleshooting. Taking the RK356x platform as an example, after powering on or resetting a Rockchip chip development board, common boot logs and critical printouts are as follows:

```
DDR Version <version information>        # DDR Stage (TPL Stage) Print
...
U-Boot SPL board init                    # Miniloader Stage (SPL Stage) Print
U-Boot SPL <version information>
...
INFO:    Preloader serial: 2             # Trust Stage (BL31 Stage) Print
NOTICE:  BL31: <version information>
NOTICE:  BL31: Built : <version information>
...
I/TC: Rockchip release version: 1.0      # OP-TEE Stage (BL32 Stage) Logging
I/TC: OP-TEE version: <version information>
...
U-Boot <version information>             # U-Boot Stage Printout
Model: Rockchip RK3568 Evaluation Board
PreSerial: 2, raw, 0xfe660000
DRAM:  2 GiB
Sysmem: init
...
Starting kernel ...
[    0.000000] Booting Linux            # Linux Kernel earlycon Printout
[    0.000000] Linux version <version information>
```

```
[    0.000000] Machine model: Rockchip RK3568 EVB2 LP4X V10 Board
[    0.000000] earlycon: uart8250 at MMIO32 0x00000000fe660000
[    0.000000] bootconsole [uart8250] enabled
...
[    0.193783] console [ttyFIQ0] enabled # Switch to fiq_debugger output
[    0.194484] bootconsole [uart8250] disabled
...
[    0.411356] Serial: 8250/16550 driver # Standard UART Initialization
[    0.412586] fe650000.serial: ttyS1 at MMIO 0xfe650000 (irq = 67, base_baud = 1500000) is
a 16550A
[    0.413461] fe690000.serial: ttyS5 at MMIO 0xfe690000 (irq = 68, base_baud = 1500000) is
a 16550A
```

1.3 io Command

The Rockchip platform typically integrates the `io` command for performing direct register read and write operations. The source code for the `io` command is located in the `external` directory of the Android SDK. The following example uses RK3568 UART5_M0 to illustrate common debugging operations using the `io` command.

Ensure that the UART5 pclk is enabled before correctly accessing the UART5 controller registers. You can directly manipulate the corresponding CRU module registers according to the TRM manual, or use the following commands:

```
echo 1 > /sys/kernel/debug/clk/pclk_uart5/clk_enable_count
```

Use the following command to view all registers of the UART5 controller:

```
io -4 -l 0x100 0xFE690000
```

Ensure that the UART5 sclk is enabled for UART5 to function properly:

```
cat /sys/kernel/debug/clk/clk_summary | grep uart5    # UART5-Related Clocks
cat /sys/kernel/debug/clk/sclk_uart5/clk_rate         # UART5 Operating Clock Frequency
echo 24000000 > /sys/kernel/debug/clk/sclk_uart5/clk_rate
cat /sys/kernel/debug/clk/sclk_uart5/clk_enable_count # UART5 Operating Clock Status
echo 1 > /sys/kernel/debug/clk/sclk_uart5/clk_enable_count
```

Ensure that the UART5_M0 IOMUX configuration is correct to obtain the corresponding signal from the target pin. The required registers must be looked up in the GRF chapter of the TRM:

```
io -4 0xFDC60310    # uart5_iomux_sel: Selects IOMUX group 0 or group 1
fdc60310:  00000000 # A value of 0 at bit 0 indicates that the UART5_M0 configuration is
correct.
io -4 0xFDC60020    # gpio2a1_sel and gpio2a2_sel, pin function selection for RX and TX
fdc60020:  00001331 # Both bits 7:4 and bits 11:8 being 3 indicate that the UART5_M0 RX and
TX configurations are correct.
```

Configure the UART5 controller Register directly to implement basic transmit and receive functionality at a baud rate of 115200:

```
io -4 0xFE690088 0x00000007 # SRR Register, used to reset UART and FIFO reset
io -4 0xFE690010 0x00000010 # Configure the UART to loopback mode via the MCR Register
io -4 0xFE69000C 0x00000080 # LCR Register, Set DLAB (Divisor Latch Access Bit) to 1 in the
LCR Register
io -4 0xFE690000 0x0000000D # DLL Register, Configure Baud Rate Divisor Coefficient
io -4 0xFE690004 0x00000000 # DLH Register, Configure Baud Rate Divisor Coefficient
io -4 0xFE69000C 0x00000003 # LCR Register, Clearing DLAB (Divisor Latch Access Bit) to 0,
and Configuring UART Protocol Parameters
io -4 0xFE690010 0x00000000 # MCR Register: Configure UART to Normal Mode
io -4 0xFE690004 0x00000001 # IER Register, Enable Receive Interrupt
io -4 0xFE690008 0x00000041 # FCR Register: Enable FIFO and set the FIFO trigger threshold
to 1/4
io -4 0xFE69009C            # SRT Register, Read/Write RX FIFO Trigger Threshold
io -4 0xFE6900A0            # STET Register, Read/Write TX FIFO Trigger Threshold
io -4 0xFE690000 0x00000055 # THR Register, transmit "U", 0x55
io -4 0xFE690000            # RBR Register: Host sends "U", reads 0x55
```

Note: When configuring DLL and DLH, first set bit 7 (DLAB (Divisor Latch Access Bit)) of the LCR Register to 1. Before performing this operation, set bit 4 (loopback) of the MCR Register to 1 to configure the UART into loopback mode. If the UART is not in loopback mode, external signal interference on the UART RX pin during DLL and DLH configuration may trigger the UART busy state, preventing bit 7 (DLAB (Divisor Latch Access Bit)) of the LCR Register from being reconfigured to 0, resulting in an error.

1.4 Disable Other Print Interference

During debugging, use the following commands to disable other print interference:

```
su
echo 0 > /proc/sysrq-trigger
```

2. Baud Rate Related

Whether the UART can operate normally at a specific baud rate primarily depends on the internal prescaler of the UART controller and the working clock prescaling strategy provided by the CRU to the UART controller. The internal prescaling operations of the UART controller are handled by the DLL and DLH Registers. The working clock provided by the CRU to the UART controller is typically named uart_sclk, which must maintain a 16x relationship with the baud rate according to the UART controller requirements. Note the following points:

- To meet accuracy requirements, please use common baud rates such as 115200, 460800, 921600, 1.5M, 3M, and 4M. Rockchip does not guarantee support for non-typical baud rates. For support of more

common baud rates within the Driver framework, please refer to the Driver code include/uapi/asm-generic/termbits.h.
- The Rockchip UART IP stably supports baud rates up to 4M. Support for higher baud rates is not guaranteed and requires corresponding modifications to the UART Driver framework code. The sign-off constraint frequency for the functional clock `uart_sclk` is typically designed to be 100MHz. Even under special use cases, the baud rate must not exceed 6M.

If the SDK version currently in use is outdated, please contact Rockchip to update the UART-related source code.

2.1 Clock Prescaling Strategy

In Linux Kernel 4.19, primarily focus on the following Driver files and function code:

```
drivers/tty/serial/8250/8250_dw.c    # dw8250_set_termios
drivers/clk/clk-fractional-divider.c # clk_fd_set_rate
drivers/clk/rockchip/clk.c           # rockchip_fractional_approximation
```

In the `dw8250_set_termios` function within `drivers/tty/serial/8250/8250_dw.c`, the 24 MHz crystal oscillator is selected as the clock source for baud rates of 1.5 Mbps and below, while the PLL, controlled by the CRU, is selected for baud rates above 1.5 Mbps. For certain special baud rates, the operating clock provided by the CRU after division may exhibit significant error. In such cases, accuracy can be improved by multiplying the requested rate by 2 (i.e., `sclk_rate = baud_rate × 16 × 2`) when the UART controller requests the operating clock from the CRU.

To reduce jitter in the operating clock input from the CRU to the UART controller, the CRU fractional divider imposes a 20x ratio limitation. However, since the UART controller contains an internal divider, the signal generally meets requirements even if this 20x limit is removed. To remove the 20x limit, refer to the `CLK_FRAC_DIVIDER_NO_LIMIT` flag in the `rockchip_fractional_approximation` function within `drivers/clk/rockchip/clk.c`. Note that some earlier SDK versions may not include the patch for this feature; please contact Rockchip FAE for assistance.

2.2 Cases

2.2.1 Incorrect Baud Rate Measurement Method

When issues arise, baud rate measurement must be performed using an oscilloscope. Only by capturing the UART TX signal waveform with an oscilloscope can the true baud rate be obtained. Measurements using logic analyzers, USB-to-UART dongles, or similar methods may not be reliable. Typically, comparing the following three values can help identify the issue:

- The actual baud rate measured from the UART TX output waveform using an oscilloscope.
- The printed operating clock frequency of the corresponding UART.
- The theoretical UART operating clock frequency calculated based on the clock division strategy.

If the printed clock frequency is normal but the measured baud rate is abnormal, verify whether the baud rate measurement process was performed correctly. If there is a discrepancy between the theoretical value derived from the clock division strategy and the actual value printed as input to the UART controller, try modifying the clock division strategy or selecting a different baud rate.

2.2.2 Errors Occur During Operation but Resolve After Restart

During operation, such as during aging tests, data transmission errors may occur after a period of time. Please verify the clock source according to the clock division strategy. If the clock source is a PLL provided by the CRU, check whether the PLL frequency changes before and after the issue occurs. The UART controller baud rate is determined by the operating clock frequency. Certain operations during use, such as suspend/resume or dynamic frequency scaling, can cause changes in the actual PLL frequency, thereby affecting the UART baud rate and resulting in baud rate deviations during data transmission.

2.2.3 Abnormal Behavior When Switching from High to Low Baud Rate

If switching from a low baud rate to a high baud rate works normally, but switching from a high baud rate to a low baud rate fails, verify whether the UART working clock source switches correctly between the CRU-provided clock and the crystal oscillator before and after the issue occurs. Typically, Rockchip uses a 24 MHz crystal oscillator. Based on the 16x relationship between the UART controller and its working clock, baud rates of 1.5 Mbps and below select the 24 MHz crystal oscillator as the clock source, while baud rates above 1.5 Mbps select the PLL clock provided by the CRU. The issue can be diagnosed by printing the UART working clock information before and after the failure.

3. Pin Multiplexing Related

A UART controller's signals usually have multiple IOMUX options for pin assignment. Configuring UART IOMUX requires setting two Registers in the GRF simultaneously. When issues occur, both Registers must be checked using the io command to ensure their values are correct:

- Select the GPIO pin function: Since a single pin may be multiplexed with functions from multiple modules, confirm that this Register is configured for the corresponding UART function.
- Select the group in the uart_iomux_sel Register: A UART controller requires overall group configuration to switch all signal pins collectively. Typically, the corresponding configuration can be found in the pinctrl code within the GRF. However, since this code is manually added, Rockchip cannot guarantee the completeness and accuracy of the IOMUX group configuration. It is necessary to confirm that this Register is set to the desired IOMUX group.

3.1 Cases

3.1.1 TX Pins of Different IOMUX Groups Can Be Used Simultaneously, But RX Pins Cannot

Rockchip designs IOMUX to optimize pin resource utilization; a single UART controller can only select one IOMUX group. Successful switching of the IOMUX group must be verified by checking the UART RX functionality. Because no selection switch is implemented in the TX pins of the UART M0 and M1 IOMUX groups, if the IOMUX function of the corresponding GPIO pins is configured for UART, both TX pins may output data. However, a selection switch exists in the RX pins of the UART M0 and M1 IOMUX groups, so both RX pins cannot simultaneously receive UART data.

3.1.2 UART Malfunction After Enabling Other Modules

Due to pin multiplexing, enabling certain modules may reconfigure IOMUX-related registers, leading to UART transmission errors. Common functions such as PWM (used for backlight in display devices), SDMMC (storage location for system boot), and JTAG (the force JTAG bit must remain disabled) can conflict with UART IOMUX if improperly used. Special attention is required when using modules that share IOMUX functions with the pins assigned to the console UART; configuration conflicts can cause print errors. When such issues occur, verify the values of the IOMUX function registers corresponding to the affected pins for troubleshooting.

4. Pin Level Issues

Incorrect UART pin level configuration can cause communication failures. To comply with the UART protocol, both RX and TX pins are typically configured with internal pull-ups. On some legacy Rockchip platforms, only the RX pin requires a pull-up, while the TX pin retains the chip's default pull-up/pull-down configuration. When issues occur, verify the following in software:

- Confirm that the pin level configurations for UART RX and TX in the kernel DTS are correct.
- Refer to the chip datasheet to verify whether the power domain IO domain configuration (1.8V or 3.0V) is correct.
- Directly measure the voltage levels on the UART RX and TX pins to ensure they are at a high logic level and the voltage values are accurate.

For hardware-related technical support, please contact Rockchip hardware engineers.

4.1 Cases

4.1.1 Hardware PCB Design Errors

If the issue persists after confirming correct software configuration, inspect the hardware PCB design, including but not limited to the following scenarios:

- RX and TX signal lines on the PCB are swapped, or incorrectly connected to low or high logic levels.
- UART flow control is enabled, and the CTS pin is forcibly pulled high by an external signal.
- Mismatched peripheral voltage levels, incorrect voltage values, or lack of leakage protection.

4.1.2 Inconsistency Between Hardware Design and Actual Chip Model

Rockchip platform chips are typically released in series, with multiple variants within the same series distinguished by model suffixes appended to the series name. Ensure that the chip datasheet used during hardware reference design matches the actual chip model deployed. To troubleshoot such issues, perform identical tests using a Rockchip-provided EVB board and monitor the pin voltage levels.

4.1.3 Peripheral TX Unable to Pull Down Main Controller RX

When the peripheral TX fails to pull down the main controller RX, preventing UART transmission initiation, first verify whether the peripheral can pull down other GPIO pins. Further investigate whether level-shifting chips exist between the peripheral and the main controller, and assess whether the peripheral pin drive strength is sufficient.

5. Interrupt-Related Issues

UART interrupts include TX transmit interrupts and RX receive interrupts. Since UART controllers typically utilize FIFOs, interrupt-related issues must be analyzed in conjunction with FIFO configurations. Focus primarily on the following Register groups:

- IER Register: Interrupt Enable Register.
- 0x80 PTIME: Programmable THRE Interrupt Mode Enable. Improves transmission interrupt efficiency by generating a THRI interrupt in advance when data in the TX FIFO triggers the TX FIFO watermark, allowing preparation of the next transmission data.
- 0x08 MSI: Modem Status Interrupt. Fourth priority; an interrupt triggered by the UART Modem, involving the MCR and MSR Registers.
- 0x04 RLSI: Receive Line Status Interrupt. Highest priority; an interrupt triggered by the UART Line, including overrun/parity/framing errors, break interrupt, and address received interrupt. Involves the LCR and LSR Registers.
- 0x02 THRI: Transmitter Holding Register Interrupt. Third priority; a transmit interrupt triggered when the TX FIFO is empty if PTIME is disabled. When PTIME is enabled, it is triggered when transmit data reaches or falls below the TX FIFO watermark.
- 0x01 RDI: Receive Data Interrupt. Second priority; a receive interrupt triggered when received data reaches or exceeds the RX FIFO watermark. Alternatively, if data exists in the RX FIFO but no new data is received within the duration of four character times, a timeout interrupt is generated.
- IIR Register: Interrupt Identification Register, used to read the sub-interrupt ID that triggered the interrupt. In addition to the four sub-interrupt types enabled in the IER Register, it includes a fifth lowest-priority Busy Detect Indication interrupt.
- FCR Register: FIFO Control Register, used to control operations such as configuring the watermarks for the RX FIFO and TX FIFO.

5.1 Cases

5.1.1 Configuring the Receive Interrupt Trigger Watermark

During UART data transmission, configuring the receive interrupt trigger watermark, specifically the RX FIFO Trigger Level in the FCR Register, is critical. The Rockchip UART RX FIFO is typically 64 Bytes or 32 Bytes and can be configured to one of the following four trigger watermarks:

- 2'b00: 1 character in the FIFO
- 2'b01: FIFO 1/4 full
- 2'b10: FIFO 1/2 full
- 2'b11: 2 characters less than full

A lower UART RX FIFO trigger watermark ensures more timely processing of received data but generates more interrupts, consuming CPU resources. The default configuration is typically FIFO 1/2 full. This can be modified by adjusting the UART_FCR_R_TRIG_10 parameter within the serial8250_do_set_termios() function in the Linux Kernel UART Driver code located at drivers/tty/serial/8250/8250_port.c. If UART receive data errors occur, try modifying the RX FIFO trigger watermark to 2'b00, which generates an interrupt for every received character.

6. DMA Related

The DMA controller on Rockchip chip platforms is typically the PL330, although some chip models use DW controllers. Please refer to the corresponding chip TRM manual for the specific DMA controller model. Unless otherwise specified, the content in this chapter applies to the PL330 DMA controller. Due to the variable-length nature of UART data transmission, UART DMA configuration strategies are unique for chips using the PL330 controller. The following points require attention:

- The default DMA transfer configuration is burst mode with a burst length of 1, meaning one byte triggers a DMA request. This UART configuration is determined by the characteristics of the PL330 controller. In UART interrupt transfer mode, the FIFO watermark in the FCR Register is default configured to 2'b10 (trigger at 1/2 FIFO), which mismatches the DMA configuration. Therefore, verify whether the FIFO watermark in the FCR Register has been reconfigured to 2'b00 (trigger per byte). For detailed methods to configure the UART FIFO trigger watermark, refer to the Interrupt-related section.
- Construct transfer data aligned to the burst length. To improve UART DMA transfer efficiency, both communication parties should strictly align data according to the burst length, padding the final packet of each transfer accordingly. Note that the burst length must match the UART FIFO trigger watermark.
- When UART operates in DMA receive mode, the receive Interrupt includes a Timeout mechanism. If the configured burst length and UART FIFO trigger watermark exceed one byte, and the final packet of each data transfer lacks redundant data padding for alignment, the last packet will be received via CPU Interrupt through the Timeout mechanism. Under this configuration, contention exists between CPU Interrupt-based data reception and DMA-requested data reception, posing a risk of data loss.

The DW DMA controller does not exhibit the aforementioned issues; however, to maintain a unified DMA controller configuration strategy, the same approach is still adopted. When using DMA, note the following:

- Verify the start and end positions of each DMA data transfer.
- Confirm the enable/disable status of the DMA controller before initiating the next DMA data transfer.

6.1 Cases

6.1.1 DMA Configurations May Differ Across Kernel Versions

This issue typically occurs when customers upgrade the system version and port DMA configuration parameters in the DTS, directly reusing previous configurations without verifying differences in DMA configurations across kernel versions. For example, in kernel 4.4, the burst mode configuration parameter is `peripherals-req-type-burst`, whereas in kernel 4.19, the burst mode configuration parameter is `arm,pl330-periph-burst`.

6.1.2 Probabilistic Data Loss During DMA-mode Reception at High Baud Rates

When using DMA to receive data at high baud rates, the RX buffer configured for DMA reception employs a circular overwrite mechanism. If the RX buffer size is too small, unretrieved data may be overwritten, leading to probabilistic data loss. This can be resolved by modifying the `rx_size` value in the `serial8250_request_dma` function within the Driver code `drivers/tty/serial/8250/8250_dma.c`, where one `PAGE_SIZE` equals 4 KB.

7. Hardware Flow Control

It is recommended to use hardware automatic flow control mode (i.e., four-wire UART) in scenarios involving high baud rates (1.5 Mbps and above) and large data volumes. Hardware automatic flow control adds RTS and CTS pins, offering higher transmission reliability compared to DMA mode. Note: Do not use software flow control when utilizing UART on Rockchip platforms.

7.1 Cases

7.1.1 Unable to Transmit Data After Enabling Hardware Flow Control

This issue is typically hardware-related and can be investigated from the following aspects:

- Incorrect hardware design or hardware faults on the development board causing the CTS pin to be forcibly pulled high.
- The counterpart's RTS pin failing to pull high or low properly, resulting in our CTS pin being continuously pulled high after connection.

8. Data Reception Errors

UART data reception errors fall into two categories: failure to receive data and reception of garbled data. Investigation can proceed from the following directions:

- Hardware level: Verify whether the RX pin is configured with an internal pull-up, check for abnormal voltage or current in the peripheral circuit, and ensure the chip pin logic level is correctly configured. If using a USB-to-serial adapter, try replacing it with a higher-quality serial cable and tool.
- Driver level: Use the `io` command to disable interrupts in the IER Register, run the reception test program, and check whether data remains in the RFL Register. Confirm that the RX FIFO trigger watermark configured in the FCR Register meets requirements.
- Application level: Verify whether the application retrieves data from the UART FIFO, whether other applications are using the same UART, whether the application's strategy for processing reported data matches the RX FIFO trigger watermark, and whether the buffer allocated by the application for UART reception meets requirements.

If the SDK version in use is outdated, contact Rockchip FAE to update the UART-related source code. Carefully read other sections of this document and combine them with the debugging methods described in *Rockchip_Developer_Guide_UART_EN* to troubleshoot issues.

8.1 Cases

8.1.1 Incorrect Data Transmission from the Counterpart

It is essential to confirm the cause of the UART error: whether it stems from incorrect data reception on our side or incorrect data transmission from the counterpart. Tools such as an oscilloscope or a USB-to-UART adapter can be used for detection. If testing with the aforementioned methods confirms that UART data reception on our side is normal, carefully verify whether the counterpart's data transmission meets the requirements.

8.1.2 Data Segmentation During Large-Volume Reception

When encountering this issue, use the UART interrupt transmission mode and verify the FIFO trigger watermark size in the FCR Register. In the UART interrupt reception mechanism, the interrupt ID read from the IIR Register distinguishes between two scenarios: interrupts triggered by the receive data watermark (4'b0100) and interrupts triggered by receive data timeout (4'b1100). These are handled separately and reported to the application layer.

Verify whether the data reporting logic in the UART Driver code meets the requirements. Pay attention to whether the value of the parameter `max_count` in the following code matches the number of loop iterations for receiving characters:

- For kernel 3.10, check `max_count` in the `receive_chars` function within `drivers/tty/serial/rk_serial.c`.
- For kernel 4.4 and 4.19, check `max_count` in the `serial8250_rx_chars` function within `drivers/tty/serial/8250/8250_port.c`.

8.1.3 Latency in Application-Layer Data Reading

Since the Linux Kernel is not a real-time system, the UART Driver reports data to the application layer via a Work Queue. If the Work Queue is blocked, it may cause latency in application-layer data reading. In kernel 3.10, the UART driver framework provides an interrupt reporting method to improve real-time performance. The modification patch is as follows:

```
diff --git a/drivers/tty/serial/rk_serial.c b/drivers/tty/serial/rk_serial.c
index 9870873c1200..ff81be78a53a 100644
--- a/drivers/tty/serial/rk_serial.c
+++ b/drivers/tty/serial/rk_serial.c
@@ -2074,6 +2074,9 @@ static int serial_rk_probe(struct platform_device *pdev)
        up->port.uartclk = 24000000;
 #endif

+       /* update recv data quickly instead of workqueue */
+       up->port.flags |= UPF_LOW_LATENCY;
+
```

In kernel 4.4 and kernel 4.19, the UART Driver framework only supports reporting data to the application layer via Work Queue. If high real-time performance is required, please use an RTOS.

9. Issues Related to Connecting Peripherals or Other Controllers

When data transmission anomalies occur with UART connections to peripherals such as Bluetooth, Wi-Fi, or NFC, or to other controllers, investigate using the following approaches:

- Determine whether the error originates from the master side or the device side: Use a protocol analyzer specific to the device, or connect two USB-to-UART dongles to the RX and TX pins of the master-side UART respectively, to capture bidirectional data during transmission. If the error is on the master side, proceed with further analysis. If the error is on the device side, contact the device supplier for collaborative resolution.
- Determine whether the erroneous data exhibits a pattern: Analyze the captured data to infer possible causes based on the location, quantity, and values of the errors.
    - If extra data causes communication failure, analyze whether a specific process sends data to this UART under special conditions. This issue frequently occurs when switching a console debug UART to a standard UART, where preceding log output under specific conditions triggers the problem.
    - If data loss causes communication failure, analyze whether another module or process has read the data, or whether the UART configuration is correct. Specifically, when using DMA mode for transmission, verify the configuration according to the instructions in the relevant DMA section.

9.1 Cases

9.1.1 Bluetooth Transmission Anomalies

When connecting a Bluetooth module via UART in scenarios requiring high baud rates (1.5M and above) and large data volumes, select a four-wire Bluetooth module with hardware flow control. Extensive testing and validation have shown that two-wire Bluetooth modules, even when using DMA mode for transmission, exhibit playback stuttering when connected to products such as Bluetooth headsets or speakers, resulting in a poor user experience. Since the transmission bottleneck primarily resides on the Bluetooth module side, optimization on the master side is not feasible.

For Bluetooth transmission anomalies, first verify whether the Firmware provided by the Bluetooth module supplier is correct. Pay particular attention to synchronizing Bluetooth Firmware updates during system version upgrades. For troubleshooting on the master side, refer to other sections of this document regarding pin voltage levels, pin multiplexing, etc. Note the following points:

- Hardware Flow Control Mode: In the DTS configuration for four-wire Bluetooth hardware flow control, the RTS pin pinctrl switching is handled in the Bluetooth node, while the CTS pin pinctrl switching is handled in the UART node. This approach is primarily determined by the suspend/resume strategy and enhances the stability of Bluetooth data reception during wake-up.
- DMA Mode: When a two-wire Bluetooth configuration is set to DMA mode, a warning regarding the absence of a Bluetooth ring buffer may occur. The DMA buffer must be configured manually according to the specific usage scenario.

9.1.2 Abnormalities When Using RS485

The Rockchip UART module natively supports only RS232, not RS485. Therefore, an external conversion module is required to use RS485. If abnormalities occur when using RS485, check the hardware circuit design and relevant voltage levels.

10. Console Printing

Rockchip console printing uses fiq_debugger. Since FIQ disables global interrupts, Rockchip employs threaded printing: print messages are first written to memory and then output. Enable the macro CONFIG_RK_CONSOLE_THREAD=y in the configuration. For related code and configuration details, refer to the document *Rockchip_Developer_Guide_UART_EN*. If the SDK version in use is outdated, contact Rockchip FAE to update the fiq_debugger source code.

When using console printing, note the following:

- The baud rate must remain consistent across all levels. For specific configuration, refer to the sections Disable All Prints or Redirect All Prints to Another UART in this document.
- If large amounts of data need to be printed, increase FIFO_SIZE in the Driver code to enhance the buffering capacity of threaded printing.

10.1 Cases

10.1.1 Shell Hangs When Sending Large Numbers of Test Commands via Console

This issue typically occurs during customer automated stress testing when large numbers of test commands are sent via the console, causing the shell to hang. This is caused by the Android shell; it is recommended to switch to a shell based on busybox or bash.

10.1.2 Carriage Return and Line Feed Issues

Console Printing typically applies special handling to the carriage return character '\r' (0x0D) and the line feed character '\n' (0x0A). Generally, when the UART outputs characters to the terminal, it checks for '\n' and inserts a '\r' character before it. When inspecting UART transmission data in ASCII format, pay special attention to the values 0x0D and 0x0A.

11. suspend, resume, and Power Consumption

Rockchip UART suspend/resume is implemented in the trust stage. A wakeup source must be added in the kernel to keep the UART operational as a wakeup source during suspend mode. Any issues arising from using the UART as a wakeup source require joint debugging of both Trust and the kernel. If such issues occur, please update to the latest Trust Firmware in the SDK. If the problem persists, contact the engineer responsible for Rockchip Trust for technical support.

If strict power consumption requirements apply during suspend, the UART used as a wakeup source must use a crystal oscillator as its working clock input to save power consumed by the CRU fractional divider.

12. Disable All Prints or Redirect All Prints to Another UART

Rockchip UART prints typically include the DDR stage, Miniloader stage, Trust Stage (Trusted Firmware-A stage), OP-TEE stage, Uboot stage, and Kernel stage. For specific print information and stage markers, refer to the Boot Log section of this document. There are two methods to disable all prints or redirect all prints to another UART:

- Use special Firmware for each stage to disable prints or redirect prints to another UART: Typically, early Rockchip platforms only support this modification method. Contact Rockchip to obtain special pre-stage Firmware support.
- Use the parameter passing mechanism to disable prints or redirect prints to another UART: Rockchip pre-stage prints use the parameter passing mechanism. Modifying the UART print parameters of the DDR Loader takes effect across all pre-stages. The Kernel stage requires separate modification.

Currently, Rockchip platforms mainly adopt the solution of using the parameter passing mechanism to disable prints or redirect prints to another UART. If the SDK version in use is an early version, the UART print function in a certain pre-stage may not have the parameter passing mechanism enabled. Contact Rockchip for technical support.

12.1 DDR Loader Modification Method

To disable or redirect prints in the DDR Loader, modify the UART print configuration in the DDR Loader by adjusting the following parameters in the file rkbin/tools/ddrbin_param.txt:

```
uart id=         # UART controller ID; setting to 0xf disables printing
uart iomux=      # Multiplexed IOMUX Pins
uart baudrate=   # 115200 or 1500000
```

For detailed usage instructions, refer to the document: `ddrbin_tool_user_guide.txt`. After modification, use the following command to regenerate the ddr.bin Firmware. Replace the specific ddr.bin Firmware name with the actual ddr.bin Firmware name used in the SDK.

```
./ddrbin_tool ddrbin_param.txt path/to/ddr.bin
```

12.2 U-Boot Modification Method

To disable printing in U-Boot, enable the CONFIG_DISABLE_CONSOLE option in menuconfig and save it to the .config file.

```
console  --->
    disable console in&out
```

In U-Boot, switching the print output is determined by the parameter passing mechanism and requires no additional modifications. The relevant code for parsing parameters in U-Boot is located in the `board_init_f_init_serial()` function within `arch/arm/mach-rockchip/board.c`.

12.3 Kernel Modification Method

12.3.1 Disabling Print Output in the Kernel

In `menuconfig`, disable the configuration option `CONFIG_SERIAL_8250_CONSOLE`.

```
Device Drivers  --->
    Character devices  --->
        Serial drivers  --->
            [ ] Console on 8250/16550 and compatible serial port
```

In the DTS configuration, locate content similar to the following and remove the UART base address and console-related configuration parameters.

```
chosen: chosen {
        bootargs = "earlycon=uart8250,mmio32";
};
```

Locate the fiq-debugger node, set serial-id to 0xffffffff, and remove the UART pin multiplexing configuration. Note that the fiq-debugger node must remain enabled to ensure the fiq-debugger process and system boot correctly.

```
fiq-debugger {
        compatible = "rockchip,fiq-debugger";
        rockchip,serial-id = <0xffffffff>;
        rockchip,wake-irq = <0>;
        /* If enabled, UART uses IRQ instead of FIQ */
        rockchip,irq-mode-enable = <1>;
        rockchip,baudrate = <1500000>;  /* Only 115200 and 1500000 */
        interrupts = <GIC_SPI 252 IRQ_TYPE_LEVEL_LOW>;
        status = "okay";
};
```

12.3.2 Switching Kernel Print Output

For example, to switch Kernel print output from UART2 to UART3, locate the following content in the DTS configuration and change the UART base address from UART2 to UART3.

```
chosen: chosen {
        bootargs = "earlycon=uart8250,mmio32,0xfe670000 console=ttyFIQ0";
};
```

Locate the `fiq-debugger` node, change `serial-id` to 3, and modify the UART3 pin multiplexing configuration. Note that you must also disable the standard UART node for UART3, since it is now being repurposed as the debug console.

```
fiq-debugger {
        compatible = "rockchip,fiq-debugger";
        rockchip,serial-id = <3>;
        rockchip,wake-irq = <0>;
        /* If enabled, UART uses IRQ instead of FIQ */
        rockchip,irq-mode-enable = <1>;
        rockchip,baudrate = <1500000>;  /* Only 115200 and 1500000 */
        interrupts = <GIC_SPI 252 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&uart3m0_xfer>;
        status = "okay";
};
```

12.3.3 Using the Original Console UART as a Standard UART

After disabling or switching the console UART as described above, the UART must be re-enabled as a standard UART. Note that in the DTS configuration workflow, the console UART must first be disabled or switched to release it before being enabled as a standard UART in the corresponding UART node.

12.4 Android Modification Method

Android requires removing the console usage in recovery; otherwise, the system will hang during a factory reset. Please modify the file `android/device/rockchip/common/recovery/etc/init.rc` and comment out the console.

```
service recovery /sbin/recovery
    #console
    seclabel u:r:recovery:s0
```

12.5 Cases

12.5.1 Issues Arising After Switching from a Logging UART to a Standard UART

If issues occur after switching a logging UART to a standard UART, first perform the same test using a different UART. If the issue does not occur on other UARTs, contact Rockchip to obtain special Firmware that disables UART logging directly in the source code for further troubleshooting. Under specific circumstances, such as DDR frequency scaling operations, switching from a logging UART to a standard UART may result in data duplication or loss.

13. Other Issues

13.1 Cases

13.1.1 Kernel Fails to Boot Normally with Certain Host Tools

Certain host serial debugging tools, such as MobaXterm and PuTTY, have software flow control (Xon/Xoff) enabled by default. During system boot, operations such as DDR frequency scaling can cause a large volume of log data to be transmitted via the UART TX pin. When this data volume exceeds the receive buffer configured on the host, software flow control is triggered, continuously sending Xoff (corresponding to the Control+S character) to the UART RX pin. On Rockchip platforms, receiving the Control+S character during the boot process triggers U-Boot Debug mode, preventing normal system startup.

The default configuration of these host tools to enable software flow control is inappropriate. Software flow control is unreliable and is not utilized on Rockchip platforms. If such issues arise, please verify the configuration of the host serial tool.

13.1.2 Changing UART Device Node Numbers

If it is necessary to change UART device node numbers, using the exchange of ttyS3 and ttyS4 as an example, modify the following content in the DTS:

```
aliases {
        serial3 = &uart4;
        serial4 = &uart3;
};
```

If swapping the node numbers between the console UART and a standard UART device, directly modify the tty_line_name() function in drivers/tty/tty_io.c to implement logic that checks and adjusts the required numbers.